1

$16 \times 16 \times 16$

2

$(3 \times 3 \times 3 + 1) = 28 \times 16 = 448$

3

$224 \times 224 \times 64$

4

$56 \times 56 \times 64$

5

模型架构

```rust
use burn::{
    nn::{
        BatchNorm, BatchNormConfig, Dropout, DropoutConfig, Linear, LinearConfig,
Relu,
        conv::{Conv2d, Conv2dConfig},
        loss::CrossEntropyLossConfig,
        pool::{MaxPool2d, MaxPool2dConfig},
    },
    prelude::*,
    tensor::backend::AutodiffBackend,
    train::{ClassificationOutput, InferenceStep, TrainOutput, TrainStep},
};

use crate::data::CatVsDogBatch;

fn block_out(size: usize) -> usize {
    let conv = (size - 1) / 2 + 1;
    (conv - 2) / 2 + 1
}

#[derive(Module, Debug)]
pub struct CatVsDogModel<B: Backend> {
    conv1: Conv2d<B>,
    bn1: BatchNorm<B>,
    conv2: Conv2d<B>,
    bn2: BatchNorm<B>,
    conv3: Conv2d<B>,
    bn3: BatchNorm<B>,
    fc1: Linear<B>,
    fc2: Linear<B>,
    dropout: Dropout,
    pool: MaxPool2d,
    activation: Relu,
}

impl<B: AutodiffBackend> TrainStep for CatVsDogModel<B> {
    type Input = CatVsDogBatch<B>;
    type Output = ClassificationOutput<B>;
```

```
    fn step(&self, batch: Self::Input) -> TrainOutput<Self::Output> {
        let item = self.forward_classification(batch.image, batch.label);
        TrainOutput::new(self, item.loss.backward(), item)
    }
}

impl<B: Backend> InferenceStep for CatVsDogModel<B> {
    type Input = CatVsDogBatch<B>;
    type Output = ClassificationOutput<B>;

    fn step(&self, batch: Self::Input) -> Self::Output {
        self.forward_classification(batch.image, batch.label)
    }
}

impl<B: Backend> CatVsDogModel<B> {
    pub fn forward(&self, input: Tensor<B, 4>) -> Tensor<B, 2> {
        let x = self.conv1.forward(input);
        let x = self.bn1.forward(x);
        let x = self.activation.forward(x);
        let x = self.pool.forward(x);

        let x = self.conv2.forward(x);
        let x = self.bn2.forward(x);
        let x = self.activation.forward(x);
        let x = self.pool.forward(x);

        let x = self.conv3.forward(x);
        let x = self.bn3.forward(x);
        let x = self.activation.forward(x);
        let x = self.pool.forward(x);

        let x = x.flatten(1, -1);
        let x = self.fc1.forward(x);
        let x = self.dropout.forward(x);
        let x = self.activation.forward(x);
        self.fc2.forward(x)
    }

    pub fn forward_classification(
        &self,
        images: Tensor<B, 4>,
        targets: Tensor<B, 1, Int>,
    ) -> ClassificationOutput<B> {
        let output = self.forward(images);
        let loss = CrossEntropyLossConfig::new()
            .init(&output.device())
            .forward(output.clone(), targets.clone());

        ClassificationOutput::new(loss, output, targets)
    }
}

#[derive(Config, Debug)]
pub struct CatVsDogModelConfig {
```

```
    pub image_size: usize,
    num_class: usize,
    hidden_size: usize,
    dropout: f64,
}

impl CatVsDogModelConfig {
    pub fn init<B: Backend>(&self, device: &B::Device) -> CatVsDogModel<B> {
        let conv1 = Conv2dConfig::new([3, 32], [3, 3])
            .with_stride([2, 2])
            .with_padding(nn::PaddingConfig2d::Explicit(1, 1, 1, 1))
            .init(device);
        let bn1 = BatchNormConfig::new(32).init(device);
        let conv2 = Conv2dConfig::new([32, 64], [3, 3])
            .with_stride([2, 2])
            .with_padding(nn::PaddingConfig2d::Explicit(1, 1, 1, 1))
            .init(device);
        let bn2 = BatchNormConfig::new(64).init(device);
        let conv3 = Conv2dConfig::new([64, 128], [3, 3])
            .with_stride([2, 2])
            .with_padding(nn::PaddingConfig2d::Explicit(1, 1, 1, 1))
            .init(device);
        let bn3 = BatchNormConfig::new(128).init(device);

        let side = (0..3).fold(self.image_size, |s, _| block_out(s));
        let fc1 = LinearConfig::new(128 * side * side,
self.hidden_size).init(device);
        let fc2 = LinearConfig::new(self.hidden_size, self.num_class).init(device);
        let dropout = DropoutConfig::new(self.dropout).init();
        let pool = MaxPool2dConfig::new([2, 2]).init();

        CatVsDogModel {
            conv1, bn1, conv2, bn2, conv3, bn3, fc1, fc2, dropout, pool,
            activation: Relu::new(),
        }
    }
}
```

各层输出尺寸（输入 $224 \times 224 \times 3$）：

层	输出尺寸	通道	参数量	说明
conv1 + bn1 + pool	56×56	32	896 + 64	stride 2 再池化
conv2 + bn2 + pool	14×14	64	18496 + 128	
conv3 + bn3 + pool	3×3	128	73856 + 256	
flatten	1152	—	—	$128 \times 3 \times 3$
fc1 + dropout(0.5) + ReLU	256	—	295168	
fc2	2	—	514	cat / dog

数据处理

```
use std::sync::Arc;

use burn::data::dataloader::batcher::Batcher;
```

```rust
use burn::data::dataset::Dataset;
use burn::data::dataset::InMemDataset;
use burn::data::dataset::transform::{PartialDataset, ShuffledDataset};
use burn::prelude::*;
use burn::tensor::{Int, TensorData};

#[derive(Clone, Copy, Debug)]
pub enum CatVsDogLabel {
    Cat = 0,
    Dog = 1,
}

#[derive(Clone, Debug)]
pub struct CatVsDogInfo {
    image: String,
    label: CatVsDogLabel,
}

impl CatVsDogInfo {
    pub fn new(image: String, label: CatVsDogLabel) -> Self {
        Self { image, label }
    }

    fn transform<B: Backend>(
        &self,
        device: &B::Device,
        size: u32,
    ) -> (Tensor<B, 3>, Tensor<B, 1, Int>) {
        let image = image::open(&self.image)
            .unwrap_or_else(|_| image::DynamicImage::new_rgb8(size, size))
            .resize_exact(size, size, image::imageops::FilterType::Triangle)
            .to_rgb8();
        let (width, height) = (image.width() as usize, image.height() as usize);
        let pixels: Vec<f32> = image
            .into_raw()
            .into_iter()
            .map(|p| p as f32 / 255.0)
            .collect();
        let image = Tensor::<B, 3>::from_data(TensorData::new(pixels, [height, width,
3]), device)
            .permute([2, 0, 1]);
        let label = Tensor::<B, 1, Int>::from_ints([self.label as i64], device);
        (image, label)
    }
}

#[derive(Clone, Debug)]
pub struct CatVsDogBatch<B: Backend> {
    pub image: Tensor<B, 4>,
    pub label: Tensor<B, 1, Int>,
}

#[derive(Clone)]
pub struct CatVsDogBatcher {
    pub image_size: u32,
}
```

```rust
impl<B: Backend> Batcher<B, CatVsDogInfo, CatVsDogBatch<B>> for CatVsDogBatcher {
    fn batch(&self, items: Vec<CatVsDogInfo>, device: &B::Device) -> CatVsDogBatch<B>
{
        let (images, labels): (Vec<_>, Vec<_>) = items
            .iter()
            .map(|item| item.transform(device, self.image_size))
            .unzip();
        CatVsDogBatch {
            image: Tensor::stack::<4>(images, 0),
            label: Tensor::cat(labels, 0),
        }
    }
}

pub fn load_pet_images(root: &str) -> InMemDataset<CatVsDogInfo> {
    let mut data = Vec::new();
    for entry in std::fs::read_dir(root).unwrap() {
        let path = entry.unwrap().path();
        if !path.is_dir() {
            continue;
        }
        let label = if path.file_name().unwrap() == "Cat" {
            CatVsDogLabel::Cat
        } else {
            CatVsDogLabel::Dog
        };
        for img in std::fs::read_dir(&path).unwrap() {
            let p = img.unwrap().path();
            if p.is_file() {
                data.push(CatVsDogInfo::new(p.to_str().unwrap().to_string(), label));
            }
        }
    }
    InMemDataset::new(data)
}

pub fn split_pet_images(
    root: &str,
    train_ratio: f64,
    seed: u64,
) -> (Arc<dyn Dataset<CatVsDogInfo>>, Arc<dyn Dataset<CatVsDogInfo>>) {
    let dataset = Arc::new(load_pet_images(root));
    let n = dataset.len();
    let n_train = (n as f64 * train_ratio) as usize;

    let train = PartialDataset::new(ShuffledDataset::new(dataset.clone(), seed), 0,
n_train);
    let valid = PartialDataset::new(ShuffledDataset::new(dataset, seed), n_train, n);
    (Arc::new(train), Arc::new(valid))
}
```

单张图的预处理链为：读入后统一转 RGB，`resize_exact` 到 224×224,逐像素除以 255 归一化到 $[0, 1]$.数据集共 24961 张，划分后训练集 19968 张、验证集 4993 张。

训练设置

优化器	Adam ($\beta_1 = 0.9, \beta_2 = 0.999, \varepsilon = 10^{-5}$)
损失函数	交叉熵
batch size	128
epochs	20
学习率	5×10^{-4}，前 156 次迭代由 5×10^{-5} 线性预热
输入尺寸	224×224
dropout	0.5
随机种子	42

模型的评估

验证集 4993 张

	预测 Cat	预测 Dog
真实 Cat	2109	361
真实 Dog	448	2075

准确率 $(2109 + 2075) \,/\, 4993 = 83.8\%$.